UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

CARRERA INGENIERIA EN INFORMÁTICA

INFORMÁTICA INDUSTRIAL

SECCIÓN 1

PROFESOR (A) CLINIA CORDERO

Informe Técnico: Logistics Optimizer (Smart Routing)

ESTUDIANTES:

V-29.630.827 BEATRIZ APONTE

V-26.513.725 Jesus Rodriguez

V-26.599.257 Luismer Ortega

MARZO 2026

1. Entorno de Ejecución y Herramientas

- Motor de Base de Datos: Neo4j v5.x ejecutado sobre Docker para garantizar un entorno de despliegue estandarizado y aislado.
- Librerías Avanzadas: Graph Data Science (GDS) para procesamiento en memoria y APOC para utilidades extendidas.
- Interfaz de Usuario: Dashboard interactivo desarrollado en Streamlit (Python), que permite la manipulación dinámica de variables (tonelaje del camión) y visualización de rutas en tiempo real.

2. Modelo de Datos Implementado

El modelo se aleja de la estructura relacional, optimizando las búsquedas mediante punteros físicos entre nodos.

- Nodos: Almacen, Interseccion, PuntoEntrega.
- Relación: CONECTA_A.
- Propiedades de Relación:
- distancia (Float)
- tiempo_estimado (Int)
- estado_trafico (Float: 0.0 a 1.0)
- limite_peso (Float)
- costo_operativo (Calculado dinámicamente)

3. Resolución de Desafíos Técnicos

- Desafío 1: Algoritmos de Grafos (GDS) Se implementó la proyección del grafo en la memoria RAM del servidor para cálculos de alta velocidad. Se utilizó el algoritmo de Dijkstra para comparar tres escenarios críticos:

- **Optimización por Distancia:** Ruta con menor longitud física.
- **Optimización por Tiempo:** Ruta más rápida considerando el tráfico actual.
- **Optimización por Costo Operativo:** Basado en la fórmula matemática requerida en el desafío 2.

- **Desafío 2: Cálculo del Costo de Ruta**

Para cumplir con la exigencia matemática del proyecto, se utilizó la función avanzada reduce de Cypher para iterar sobre los tramos de la ruta y calcular el costo total acumulado.

Fórmula:

Costo= $\sum_{i=1}^{n}(\mathrm{distancia_i} \times \mathrm{factor_trafico_i})$

Evidencia de validación: El sistema reporta valores precisos (ej. 59.967) tras procesar la sumatoria de aristas en rutas complejas de múltiples saltos.

```
neo4j$ MATCH ruta = (o:Almacen {id: 'A1'})-[calles:CONECTA_A*1..10]→…
```

Table | Text | Code

	Camino	Costo_Total_Matematico
1	["A1", "I25", "I41", "P7", "I9", "I19", "I16", "I26", "I10", "I5", "P1"]	59.967

Started streaming 1 records after 1605 ms and completed after 2130 ms.

Desafío 3: Restricciones de Carga Dinámicas

Se implementó una técnica de Cypher Projections (Grafos Anónimos).

El sistema recibe el peso del camión desde la interfaz.

Se genera una proyección de grafo "al vuelo" que filtra y elimina de la memoria RAM todas las relaciones CONECTA_A cuyo limite_peso sea inferior al peso del vehículo.

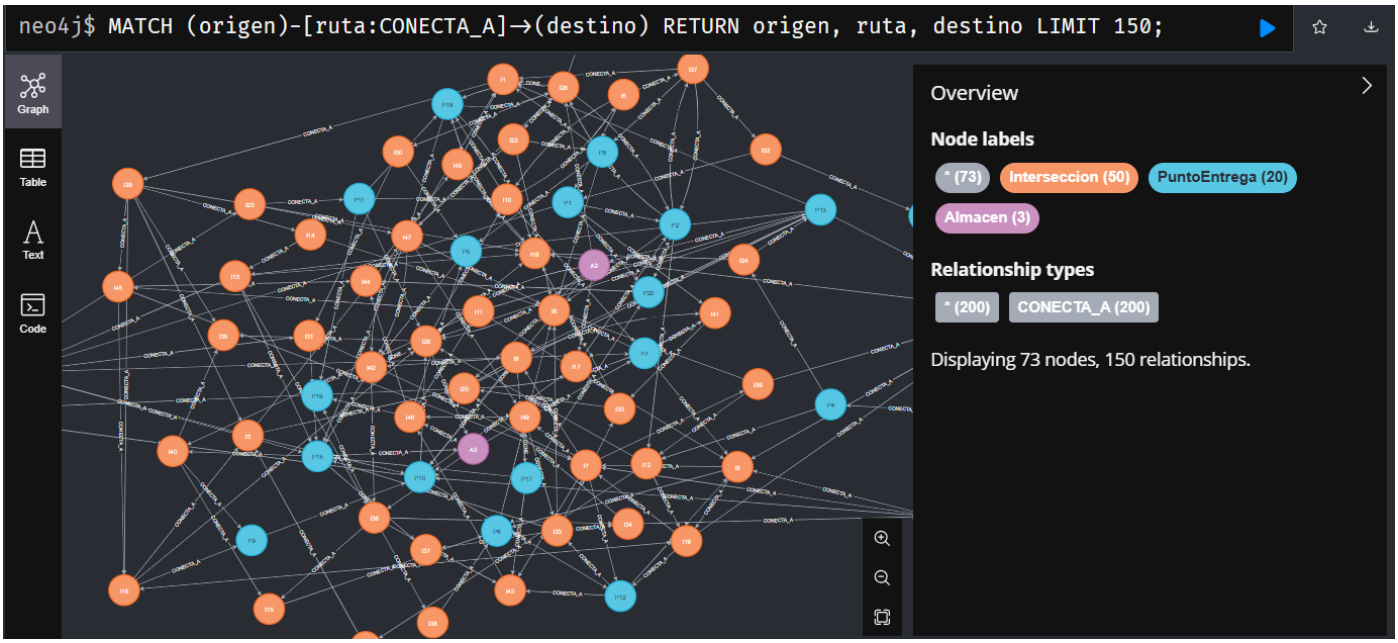
Si no existen rutas válidas tras el filtrado, el sistema lanza una excepción de "Ruta Bloqueada".

4. Justificación de Neo4j vs SQL

La superioridad de Neo4j en este caso radica en la Adyacencia sin Índices. Mientras que un motor SQL requeriría múltiples JOINs para calcular una ruta entre 50 intersecciones (degradando el rendimiento), Neo4j navega punteros de memoria en tiempo constante O(1) por salto. Esto permite que el filtrado de aristas por peso y el cálculo de costo con reduce ocurran en milisegundos sobre el grafo proyectado.

5. Evidencias de Implementación (Anexos)

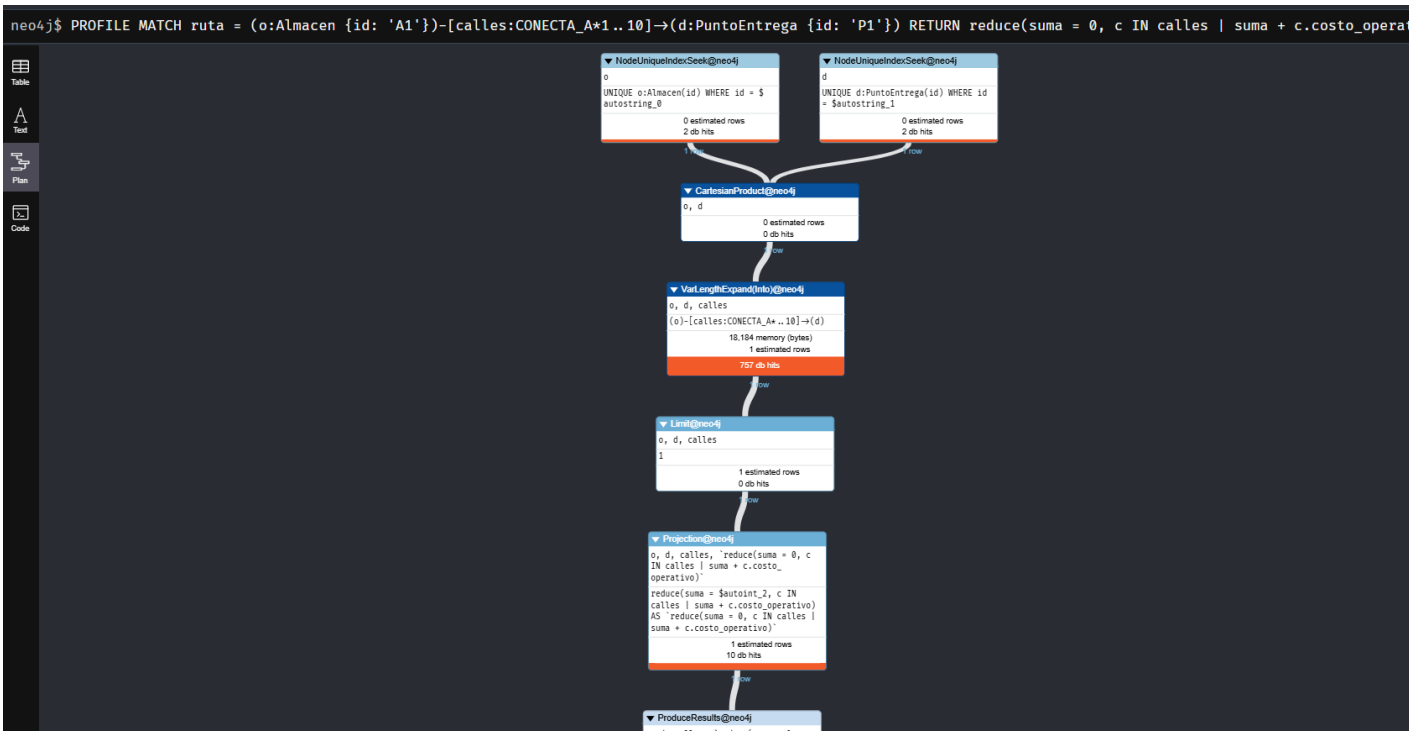
Captura 1 (Telaraña de Nodos): Demostración del volumen de datos generado por el script de carga masiva.



Captura 2 (Análisis de Rendimiento y Optimización): Evidencia de la creación de restricciones de unicidad (índices) y el plan de ejecución usando PROFILE. Se demuestra un acceso eficiente a los nodos mediante NodeUniqueIndexSeek, evitando escaneos totales de la base de datos.

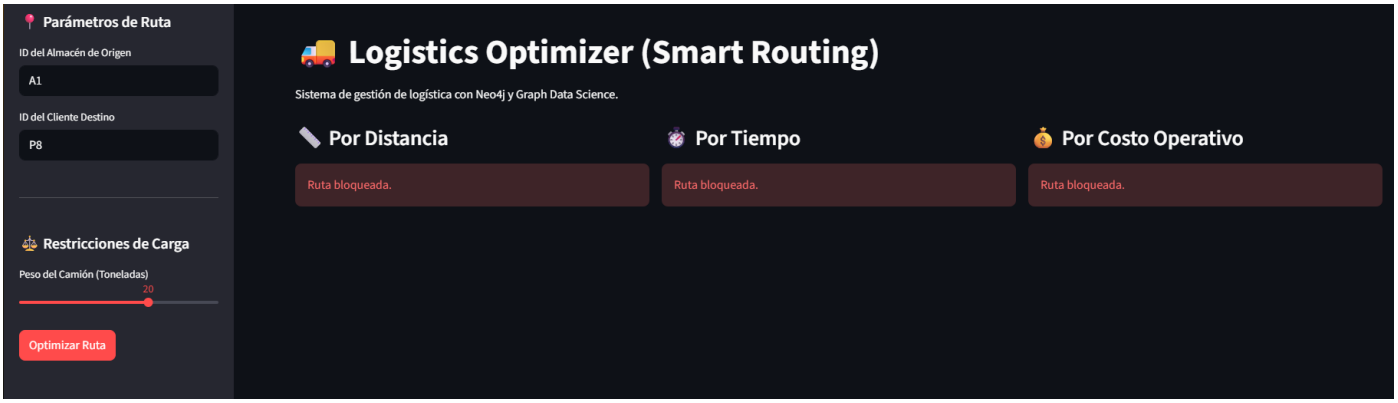
```
$ CREATE CONSTRAINT unico_almacen IF NOT EXISTS FOR (a:Almacen) REQUIRE a.id IS UNIQUE; CREATE CONSTRAINT unico_punto IF NOT EXISTS FO

neo4j$ CREATE CONSTRAINT unico_almacen IF NOT EXISTS FOR (a:Almacen) REQUIRE a.id IS UNIQUE
neo4j$ CREATE CONSTRAINT unico_punto IF NOT EXISTS FOR (p:PuntoEntrega) REQUIRE p.id IS UNIQUE
neo4j$ CREATE CONSTRAINT unico_inter IF NOT EXISTS FOR (i:Interseccion) REQUIRE i.id IS UNIQUE
```

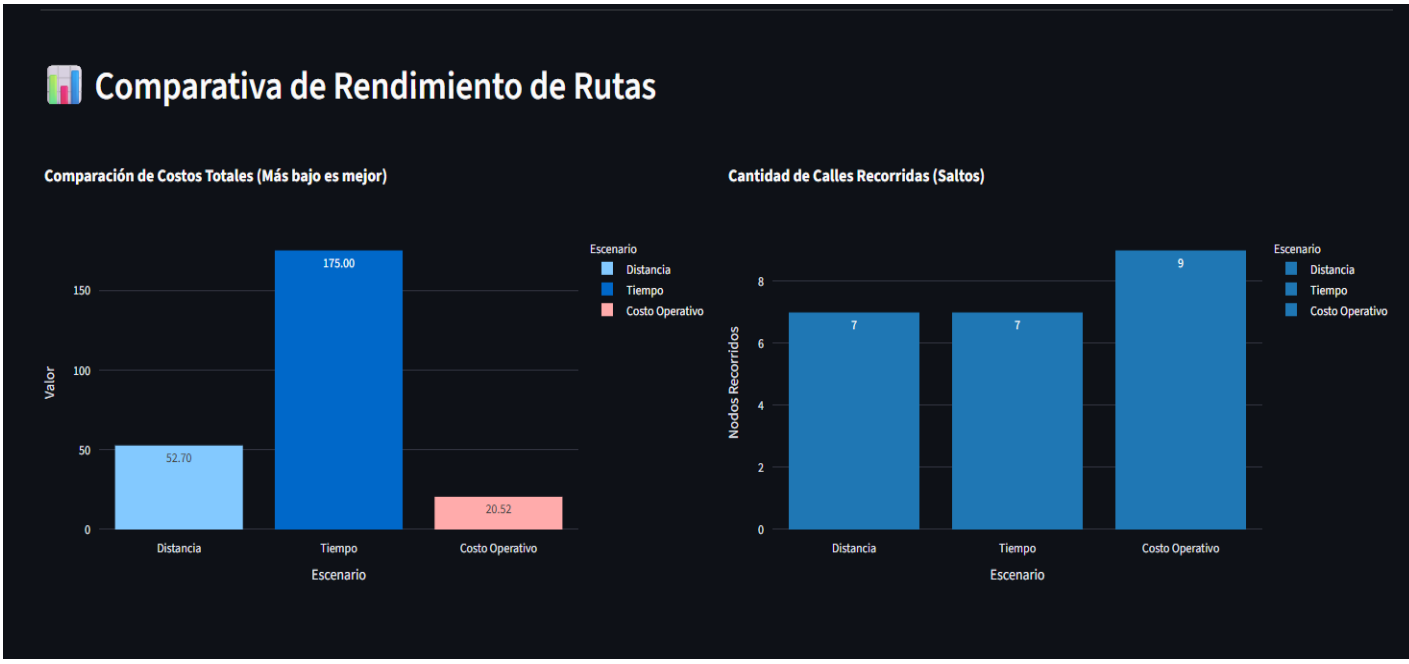


Captura 3 (Dashboard Exitoso): Comparativa de las 3 rutas entre A1 y P8 con camión de 10t.

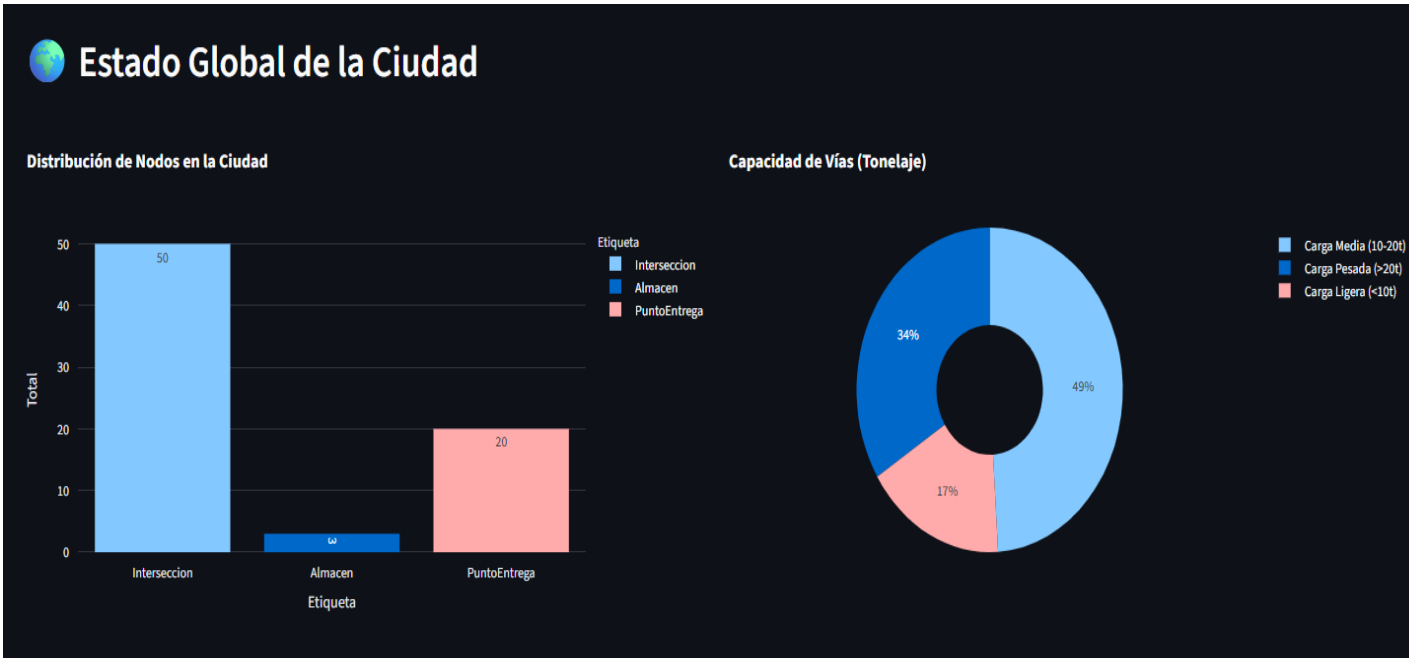
Captura 4 (Dashboard Bloqueado): Validación del filtrado de aristas cuando el camión excede la capacidad de las vías.



Capture 5 (Graficos dinamicos): Se evidencia la comparación de los costos y cantidad de calles recorridas



Capture 6 (Graficos estaticos): Distribucion de nodos en la ciudad y capacidad de las vías



6. Conclusiones y Logros Técnicos

La implementación de **Logistics Optimizer** demostró el poder de las bases de datos orientadas a grafos para resolver problemas de enrutamiento complejos. A través de la librería Graph Data Science (GDS), se superó el límite estático de las consultas tradicionales al proyectar la red de la ciudad directamente en la memoria RAM.

El mayor logro técnico del proyecto fue la aplicación de **Cypher Projections** (Grafos Anónimos) para cumplir con las restricciones de carga. Esto permitió que la aplicación no solo encontrara la ruta más corta usando Dijkstra o calculara el costo operativo mediante la función de agregación REDUCE, sino que reaccionara dinámicamente a las variables del usuario.

Al filtrar en milisegundos las calles de la ciudad que no soportaban el tonelaje del camión, se logró una simulación logística robusta, evitando que el sistema arrojara rutas inviables en vías restringidas. En conclusión, Neo4j demostró ser la tecnología ideal para la toma de decisiones espaciales, superando con creces la rigidez algorítmica de las bases de datos relacionales tradicionales.

Repositorio (Rama Main):

https://github.com/AlgoBea/SmartRoute-DB-Project.git